

Bryan Wang  
Ms. Kaur  
CS 314H  
September 4, 2026

## Program 1 Report

My name is Bryan Wang, and today I will be reporting on my progression through and learning from Project 1 of CS 314H. I will be dividing this report into 4 sections: (1) Overview, (2) Solution Design, (3) Assignment Discussion/Thoughts, and (4) Software Testing Methodology.

### Section 1: Overview

The assigned goal of Project 1 was to implement several methods to manipulate digital images, then use those methods to produce interesting images and write a report on it. Personally, my goals for this project were to reconfigure my IDE (VS Code), review java and get familiar with it again (haven't used it in a year), and learn how CS 314H projects are designed and worked out.

### Section 2: Solution Design

Most of my solutions followed the same basic logic: iterating through the pixels 2d array and modifying pixels at each index traversed. To do this, I

1. saved the length of pixels into int variable height
2. iterated my first (outer) loop from index 0 through height-1 saving the variable as y
3. iterated my second (inner) loop from index 0 through the length-1 of the pixels array at index y

Some of my solutions involved the use of a custom helper function I wrote called `jaggedCheck()`, which would take a 2d int array and a Boolean (square) as inputs and return a 2d array. This function was designed to help convert any jagged 2d arrays into rectangular 2d arrays (where every array within the 2d array has the same length) for functions where my solutions wouldn't work with jagged arrays, such as `Grow()` or `Shrink()`. I wrote this helper function rather early in the project, and added the square Boolean because I thought I might need a square array for certain problems later on (which I ended up not needing). The function works as follows:

1. save the length of the 2d array (pixels) as an int (height) and create an int width set at 0
2. iterate from 0 to height-1 as y and save width as the max between width and the length of the array at pixels index y
3. if square is true (if we want a square array), set height and width equal to the max of both
4. create a new 2d array with length equal to height and each array's length equal to width
5. iterate through the new array. At each index, if the index exists at the original input array, copy the value from the original to the new array. If not, create a new pixel at that index set to the color black
6. return the new array

The function fills in unassigned values with black, which may cause some distortion on effects such as Shrink and Smooth()

For the first 6 functions (NoRed, NoGreen, NoBlue, RedOnly, GreenOnly, and BlueOnly), I iterated through the input 2d array and created a new pixel at each index, using the existing pixel's RGB values and modifying some to 0 as needed.

For BlackAndWhite, I iterated through the input 2d array and created an int variable color at each array, adding up the red, green, and blue values of that pixel and dividing it by 3. Then I checked if the average was less than 128 (colors closer to black are 0-127, closer to white are 128-255), set the color variable to 0 (black) if it was, and set it to 255 (white) if it wasn't. Then I created a new pixel using the color variable for RGB. Essentially, for every pixel this function checks if its RGB values sum to less than 384 (0-383) and sets the color to black if so, otherwise (384-768) it sets it to white. One limitation of this approach is that it isn't perfectly balanced, the median sum value (384) is included by white so there will be slightly more white pixels than black on average.

For Horizontal and Vertical reflect, I first used jaggedCheck() to normalize the 2d array. Then, I only iterated through half of the image depending on which it was (horizontal iterated half of x, vertical iterated half of y) to avoid double swapping. For every pixel, I would save it to a temporary temp int variable, then replace the existing pixel with a pixel at the opposite end (for vertical, one at the 2d array index height - 1 - y and for horizontal, one at the 1d array index (1d array length - 1 - x). I subtract 1 because arrays are 0 indexed, so every index value is 1 less than what it would be if I just subtracted the current index from the end.

For Grow, I first used `jaggedCheck()` to normalize the 2d array, then created a copy of the normalized 2d array with twice the length and twice the width (1d array length). When iterating through the larger array (using y and x in the same way as previous problems), for every pixel, I would map the original pixel to a 2x2 grid in the new larger array, starting from the top left (at index[2\*y][2\*x]) and copying it to the other 3 corners. I did this by iterating through 2 more nested loops storing variables i and j from 0 to 1, adding those variables to 2\*y and 2\*x respectively to complete the 2x2 grid for each pixel.

For Shrink, I approached this problem in a similar way to how I approached Grow. I would split the 2d array into 2x2 plots (starting from 0,0), average each plot into one value, and then place that new value in a smaller final array. Since plots with odd dimensions cannot be simplified into these 2x2 plots, I would discard additional rows/columns as needed to make the dimensions even. I started off by using `jaggedCheck()` to normalize the 2d array, then shrank the modified array if not even to make it even. Then I created a copy of the modified array with half the width and length. Since I was starting the plots from (0,0), I iterated through the bottom right corners of each plot starting from (1,1), and increasing each step by 2 each time. This ensured that I wouldn't go out of bounds, since I would only be using indexes less than the bottom right. Instead of using more nested loops like I did in grow, I manually averaged each of the red, green, and blue values of the pixels in each 2x2 and created a new pixel in the new array at half the y and x indexes with these averaged values.

For Threshold, I first created placeholder red, green, and blue variables. Then, I iterated through the array (y then x), and for each pixel's red, green, and blue values, checked if it was greater than or equal to 128, setting the appropriate placeholder to 255 if so and 0 if not. Then I'd make a new pixel at that index with the placeholder values. Since every index's operation was independent from other indexes, there was no need to use a normalized 2d array. All I had to do was use each 1d array's own length in the inner loop when iterating to make sure I don't go out of bounds on jagged arrays.

For Smooth, I first normalized the array using `jaggedCheck()`, then created a copy of the normalized array called `pixels2`. When iterating through the normalized array, I kept a running sum for each color as well as a counter for how many pixels I used. I would then try to get the pixels directly around the original pixel in a 3x3 grid by checking if their index would be within bounds. If it was, I would add its r/g/b values to the respective counters and increase the division counter by 1. Once I'd checked all 8 pixels around the original, I would divide each running sum by the division counter and create a new pixel in the new array with those color averages. The division counter would prevent me from dividing by more than the number of colors I got when averaging out side or corner pixels.

All of these functions assume that the input 2d array pixels contains arrays of at least length 1 (no empty arrays or nulls) filled with valid RGB values stored as integers.

### Section 3: Assignment Discussion/Thoughts

Overall, I thought this was a great project that introduced me to what CS 314H projects are like and helped me refresh my java skills, all while being slightly tricky but not too difficult.

One of the first major challenges I encountered happened before I even started the project, having to do with setting up VS Code. I haven't used VS Code for java in a long time, and while downloading the required extensions for this class, I ran into a problem with one of my extensions failing to load and causing VS Code to flag lots of non-existent errors. I spent almost 2 hours trying to debug it before realizing that one of the new extensions was conflicting with an old one that I used (Language Support for Java by Red Hat was conflicting with Java Language Support by George Fraser), and deleting the old extension resolved the issue.

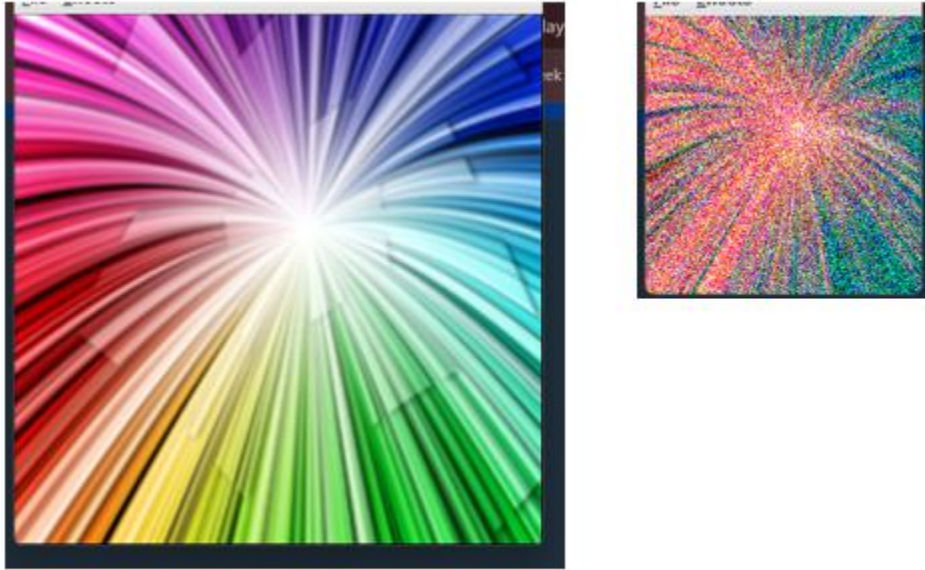
The first function I had a challenge with was BlackAndWhite. My first implementation took the average of the RGB values at each pixel and applied that constant average to each R/G/B parameter, which ended up making the image grayscale. I initially confused greyscale with black and white, but while writing test cases for this function I caught my mistake and fixed it.

My first solution for Vertical and Horizontal Reflect involved creating a copy of the original array and assigning each spot in the new array with it's reflection in the original. But while writing test cases for it, I realized that this wouldn't work with jagged arrays and created the jaggedCheck() helper function to resolve that. During this I also realized that using a temporary variable to store the function would be more time efficient, and after some testing I realized that I would have to iterate though only half the array to prevent double swapping.

Grow was relatively straightforward. Shrink was more difficult, at first I thought it only worked with square arrays but realized that instead it only worked with even dimension arrays. When writing test cases for shrink, I kept running into an array out of bounds error. I spent a long time trying to debug the Shrink function itself before realizing that I was iterating through the original array's dimensions in the test case rather than the shrunken array.

Smooth was the most difficult function for me. At first I divided each average by 9, but when testing images I realized that pictures would get more dark the more I smoothed them, so I

added the division counter. I also found an interesting result where whenever I would smooth an image and then shrink it, the colors would get distorted (see below).



After debugging for a while, I realized that the problem was in my Shrink function, not my Smooth function. I was directly adding all the int RGB values when averaging them, instead of averaging the r/g/b components separately and creating a new pixel with those averages. Once I changed that the images went back to normal

I put screenshots of a few interesting transformations below:

If I want to maximize the red, green, or blue value of every pixel in an image, I can remove that color from the image and then invert it. But this still leaves the image inverted. To get back to the original with the color maximized, I can just do the process again.



Left: Original | Middle: NoRed → Invert | Right: NoRed → Invert → NoRed → Invert



NoGreen → Invert → NoGreen → Invert

I thought my solutions were ok, they account for some major edge cases (ex. jagged arrays) while not fully checking for some others (ex. null arrays, hopefully I don't get points off for that).

#### Section 4: Software Testing Methodology

When testing my functions, I would first run them through several of the provided test images to see if they generally functioned in a way similar to how I intended. I'd also use them in combination with other functions to see if there were some niche cases where they wouldn't work (like with the shrink + smooth bug).

After I finished writing and testing all of my functions, I started writing test cases. I could use the test cases to find small inconsistencies (such as the black and white vs greyscale or the smoothing average error) and play around with niche test cases that I probably wouldn't find in photos. It was here that I tested and modified my jaggedCheck() function for jagged arrays.